

BÀI THỰC HÀNH STATE PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Frontend: React (gọi API, chuyển trạng thái đơn hàng)
 - ✓ Áp dụng State Pattern đúng chuẩn GoF
 - ✓ Thể hiện rõ việc **object thay đổi hành vi** khi state thay đổi
-

BÀI TOÁN

Hệ thống E-commerce quản lý trạng thái đơn hàng:

-  Pending (Chờ xử lý)
-  Processing (Đang xử lý)
-  Shipped (Đã giao hàng)
-  Cancelled (Đã hủy)

 Yêu cầu:

- Đơn hàng **không dùng if-else để xử lý trạng thái**
 - Mỗi trạng thái có hành vi riêng
 - Chuyển trạng thái phải đúng rule
-

KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
  ↓
POST /api/orders/action
  ↓
OrderService
  ↓
Order (Context)
  ↓
IOrderState (State)
  ↓
PendingState | ProcessingState | ShippedState |
CancelledState
```

BACKEND – ASP.NET Core 8 Web API

1 Tạo project

```
dotnet new webapi -n OrderStateDemo
cd OrderStateDemo
```

Cấu trúc thư mục

```
OrderStateDemo
├── Controllers
│   └── OrdersController.cs
├── Services
│   └── OrderService.cs
├── States
│   ├── IOrderState.cs
│   ├── PendingState.cs
│   ├── ProcessingState.cs
│   ├── ShippedState.cs
│   └── CancelledState.cs
├── Models
│   ├── Order.cs
│   └── OrderActionRequest.cs
└── Program.cs
```

1. STATE INTERFACE

States/IOrderState.cs

```
using OrderStateDemo.Models;
namespace OrderStateDemo.States;

public interface IOrderState
{
    string Name { get; }

    void Next(Order order);
    void Cancel(Order order);
}
```

2. CONTEXT (ORDER)

Models/Order.cs

```
using OrderStateDemo.States;

namespace OrderStateDemo.Models;
```

```

public class Order
{
    public int Id { get; set; }

    public IOrderState State { get; private set; }

    public Order()
    {
        State = new PendingState();
    }

    public void SetState(IOrderState state)
    {
        State = state;
    }

    public void Next()
    {
        State.Next(this);
    }

    public void Cancel()
    {
        State.Cancel(this);
    }

    public string GetStateName() => State.Name;
}

```

✿ 3. CONCRETE STATES

🌀 PendingState.cs

```

using OrderStateDemo.Models;
namespace OrderStateDemo.States;

public class PendingState : IOrderState
{
    public string Name => "Pending";

    public void Next(Order order)
    {
        order.SetState(new ProcessingState());
    }
}

```

```
    public void Cancel(Order order)
    {
        order.SetState(new CancelledState());
    }
}
```

● ProcessingState.cs

```
using OrderStateDemo.Models;

namespace OrderStateDemo.States;

public class ProcessingState : IOrderState
{
    public string Name => "Processing";

    public void Next(Order order)
    {
        order.SetState(new ShippedState());
    }

    public void Cancel(Order order)
    {
        order.SetState(new CancelledState());
    }
}
```

● ShippedState.cs

```
using OrderStateDemo.Models;

namespace OrderStateDemo.States;

public class ShippedState : IOrderState
{
    public string Name => "Shipped";

    public void Next(Order order)
    {
        throw new Exception("Order already shipped");
    }

    public void Cancel(Order order)
    {

```

```
        throw new Exception("Cannot cancel shipped  
order");  
    }  
}
```

● CancelledState.cs

```
using OrderStateDemo.Models;  
namespace OrderStateDemo.States;  
  
public class CancelledState : IOrderState  
{  
    public string Name => "Cancelled";  
  
    public void Next(Order order)  
    {  
        throw new Exception("Cancelled order cannot  
proceed");  
    }  
  
    public void Cancel(Order order)  
    {  
        throw new Exception("Order already cancelled");  
    }  
}
```

✿ 4. ORDER SERVICE

Services/OrderService.cs

```
using OrderStateDemo.Models;  
  
namespace OrderStateDemo.Services;  
  
public class OrderService  
{  
    private static readonly Dictionary<int, Order>  
_orders = new();  
  
    public Order CreateOrder()  
    {  
        var order = new Order  
        {  
            Id = _orders.Count + 1  
        };  
    }  
}
```

```

        _orders[order.Id] = order;

        return order;
    }

    public Order GetOrder(int id)
    {
        return _orders[id];
    }

    public string Next(int id)
    {
        var order = _orders[id];
        order.Next();
        return order.GetStateName();
    }

    public string Cancel(int id)
    {
        var order = _orders[id];
        order.Cancel();
        return order.GetStateName();
    }
}

```

5. REQUEST MODEL

Models/OrderActionRequest.cs

```

namespace OrderStateDemo.Models;

public class OrderActionRequest
{
    public int OrderId { get; set; }
    public string Action { get; set; } // next | cancel
}

```

6. CONTROLLER

Controllers/OrdersController.cs

```

using Microsoft.AspNetCore.Mvc;
using OrderStateDemo.Models;
using OrderStateDemo.Services;

```

```

namespace OrderStateDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _service;

    public OrdersController(OrderService service)
    {
        _service = service;
    }

    [HttpPost("create")]
    public IActionResult Create()
    {
        var order = _service.CreateOrder();
        return Ok(order);
    }

    [HttpPost("action")]
    public IActionResult Action(OrderActionRequest
request)
    {
        try
        {
            string state = request.Action.ToLower()
switch
            {
                "next" => _service.Next(request.OrderId),
                "cancel" =>
_service.Cancel(request.OrderId),
                _ => throw new Exception("Invalid
action")
            };

            return Ok(new { state });
        }
        catch (Exception ex)
        {
            return BadRequest(ex.Message);
        }
    }
}

```

7. ĐĂNG KÝ DI

Program.cs

```
using OrderStateDemo.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddSingleton<OrderService>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();
app.UseCors("AllowReact");
app.MapControllers();

app.Run();
```

CHẠY BACKEND

```
dotnet run
```

FRONTEND – REACT

Tạo project

```
npx create-react-app order-ui
```



```
cd order-ui
npm install axios
```

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [orderId, setOrderId] = useState(null);
  const [state, setState] = useState("");

  const createOrder = async () => {
    const res = await
    axios.post("https://localhost:xxxx/api/orders/create");
    setOrderId(res.data.id);
    setState(res.data.state?.name || "Pending");
  };

  const action = async (actionType) => {
    const res = await axios.post(
      "https://localhost:xxxx/api/orders/action",
      {
        orderId: orderId,
        action: actionType,
      }
    );

    setState(res.data.state);
  };

  return (
    <div style={{ padding: 40 }}>
      <h2>Order State Demo</h2>

      <button onClick={createOrder}>Create Order</button>

      <h3>Order ID: {orderId}</h3>
      <h3>State: {state}</h3>

      <button onClick={() => action("next")}>Next
State</button>
      <button onClick={() =>
action("cancel")}>Cancel</button>
    </div>
  );
}
```

```
}  
export default App;
```

▶ CHẠY FRONTEND

```
npm start
```

🔗 LUỒNG XỬ LÝ (STATE PATTERN)

Khi gọi **Next**

1. Order đang ở **Pending**
2. Gọi:

```
order.Next()
```

3. Thực chất:

```
PendingState.Next()
```

4. → chuyển sang `ProcessingState`
-

Khi gọi **Cancel**

- Nếu `Pending` → `Cancelled`
 - Nếu `Processing` → `Cancelled`
 - Nếu `Shipped` → ❌ `Exception`
-

🔗 ĐIỂM QUAN TRỌNG (GIẢNG DẠY)

❌ Cách sai (anti-pattern)

```
if (state == "Pending") ...  
else if (state == "Processing") ...
```

✅ **State Pattern**

- Mỗi state là 1 class
- Logic nằm trong state

- Context (Order) KHÔNG biết logic

MỞ RỘNG BÀI TẬP

Yêu cầu sinh viên:

1. Thêm trạng thái:
 - Returned
 - FailedDelivery
2. Log lịch sử state
3. Lưu DB (EF Core)
4. Vẽ UML State Diagram